

Enterprise Architecture

Solution Architecture Standards

Standards, Architecture Patterns and Technology Guidelines

DOC-SOL-ARCH-STD V-1.4

Effective Date: 15/03/2026

SENSITIVITY CLASSIFICATION: Internal Use

Table of Contents

- Revision History 3
- 1. Introduction 3
 - 1.1 Purpose 3
 - 1.2 Scope 3
 - 1.3 Enterprise Context 3
- 2. Architecture Governance 4
 - 2.1 Architecture Decision Records 4
 - 2.2 Reference Architecture Alignment 4
- 3. Design Principles 5
 - 3.1 Loose Coupling and High Cohesion 5
 - 3.2 API-First Design 5
 - 3.3 Statelessness and Horizontal Scalability 5
 - 3.4 Resilience by Design 6
- 4. Application Architecture 6
 - 4.1 Containerisation and Packaging 6
 - 4.2 Front-End Architecture 6
- 5. Integration Architecture 7
 - 5.1 Event-Driven Integration 7
- 6. Non-Functional Requirements 7
 - 6.1 Observability 7
 - 6.2 Availability and Recovery 7
- 7. Technology Reference Model 8
- Appendix A: Glossary of Terms 8

Revision History

Version	Date	Changed By	Description
1.0	2024-01-22	J. Whitfield	Initial publication
1.4	2026-03-15	J. Whitfield	Observability and NFR sections expanded

1. Introduction

1.1 Purpose

This document defines the Solution Architecture Standards that govern how applications and technology solutions are designed, integrated and deployed. Alongside each standard it states the recommended patterns and reference technologies, so that teams have both the rule and the means of implementing it.

1.2 Scope

These standards apply to all new solution designs, major enhancements and technology selections, and are binding for internal teams as well as external vendors delivering solutions on the organisation's behalf. Every significant design must be reviewed by the Architecture Review Board before implementation.

1.3 Enterprise Context

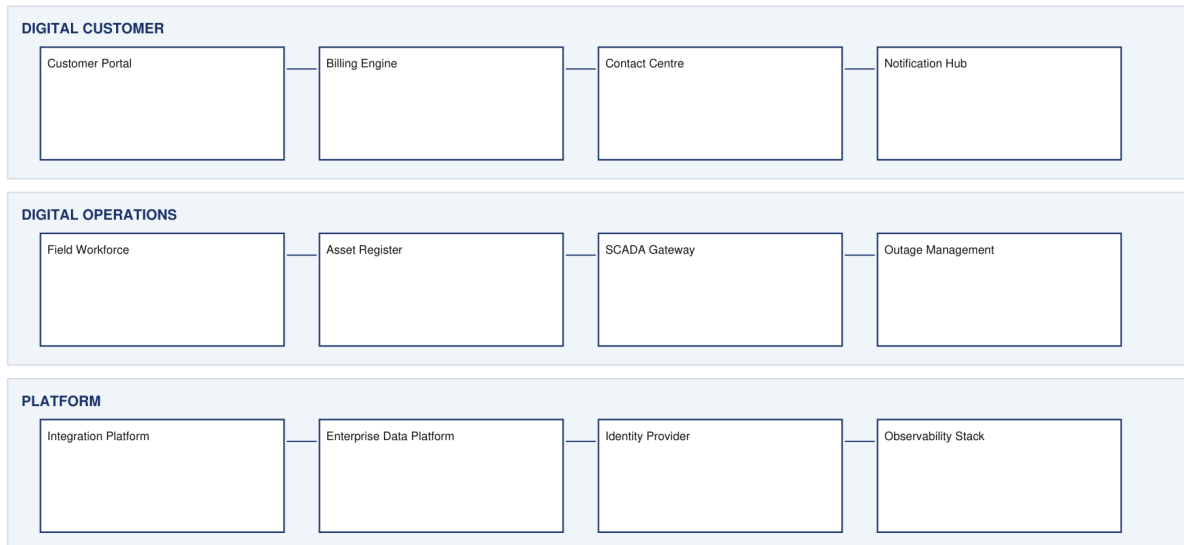
ENTERPRISE CONTEXT

Figure 1 · Enterprise context. Systems are grouped by domain; arrows indicate governed data flow through the integration platform.

2. Architecture Governance

2.1 Architecture Decision Records

Statement: Significant architecture decisions, including rationale, alternatives considered and trade-offs, must be documented as Architecture Decision Records and retained alongside the solution.

Rationale: Undocumented decisions are quickly forgotten and frequently relitigated.

Requirements:

- ADRs must be created for decisions with long-term or cross-team impact
- ADRs must record status: proposed, accepted, deprecated or superseded
- ADRs must be version-controlled alongside the solution's source code

Applicable Patterns / Technology Guidance:

- Use the lightweight ADR template rather than a heavyweight document format

2.2 Reference Architecture Alignment

Statement: New solutions must be designed against an applicable reference architecture where one exists, with deviations explicitly justified.

Rationale: Reference architectures encode proven, pre-vetted decisions and reduce design time.

Requirements:

- Deviations must be documented with rationale and reviewed by architecture governance
- Exceptions must be time-bound and recorded in the exception register

3. Design Principles

3.1 Loose Coupling and High Cohesion

Statement: Components must be designed with well-defined interfaces and minimal interdependency, so each can change independently.

Rationale: Tightly coupled systems are expensive to change and test.

Requirements:

- Components must communicate through versioned interfaces rather than direct database access
- Shared databases across independently owned services must be avoided

Applicable Patterns / Technology Guidance:

- Event-driven architecture and publish/subscribe messaging
- API Gateway pattern to abstract internal topology

3.2 API-First Design

Statement: Application capabilities must be exposed through documented, versioned APIs designed contract-first, before any user interface is built against them.

Rationale: Contract-first design enables parallel development and treats the API as a durable product rather than a by-product.

Requirements:

- APIs must be designed contract-first using an agreed specification format such as OpenAPI
- APIs must maintain backward compatibility within a major version
- APIs must be registered in the API catalogue or gateway for discoverability

Applicable Patterns / Technology Guidance:

- OpenAPI 3.x for contract-first specification
- API Gateway for authentication, throttling and traffic management

3.3 Statelessness and Horizontal Scalability

Statement: Application components must be stateless wherever possible, with session and state externalised to a dedicated store, to support horizontal scaling.

Rationale: Stateless components scale out by adding instances and are resilient to individual instance failure.

Requirements:

- Session state must be externalised to a shared cache or data store rather than held in local application memory
- Components must be safely restartable without loss of in-flight business state
- Horizontal scaling must be the default; vertical scaling requires justification in an ADR

Applicable Patterns / Technology Guidance:

- Distributed cache such as Redis for externalised session state
- Load balancer with least-connections distribution across stateless instances

3.4 Resilience by Design

Statement: Solutions must anticipate the failure of dependencies and degrade gracefully rather than propagating failure across the system.

Rationale: In distributed systems, dependency failure is a matter of when, not if.

Requirements:

- Timeouts and retries with exponential backoff must be applied to all external calls
- Critical dependencies must have a defined fallback or degraded mode
- Unprocessable asynchronous messages must be routed to a dead-letter queue rather than discarded

Applicable Patterns / Technology Guidance:

- Circuit Breaker pattern
- Bulkhead pattern to isolate resource pools
- Retry with exponential backoff and jitter

4. Application Architecture

4.1 Containerisation and Packaging

Statement: Application components must be packaged as portable, self-contained container images that include their runtime dependencies.

Rationale: Containerisation ensures consistency between development, test and production.

Requirements:

- Images must be built from an approved base image
- Images must be scanned for vulnerabilities before deployment
- Images must be immutable; configuration must be externalised rather than baked in

Applicable Patterns / Technology Guidance:

- Docker or an equivalent OCI-compliant runtime
- Kubernetes for orchestration
- Container registry with vulnerability scanning

4.2 Front-End Architecture

Statement: User-facing applications must use a component-based front-end architecture with a clear separation from back-end business logic, accessed exclusively through APIs.

Rationale: Separating concerns allows each to evolve independently and supports multiple front ends over the same services.

Requirements:

- Front-end applications must not embed business rules belonging in the service layer
- Front-end applications must consume back-end capabilities only through published APIs

Applicable Patterns / Technology Guidance:

- Component-based frameworks such as React, Angular or Vue
- Backend-for-Frontend where aggregation is required

5. Integration Architecture

5.1 Event-Driven Integration

Statement: Where multiple systems react to the same business event, solutions must publish to a shared event backbone rather than each producer calling each consumer.

Rationale: An event-driven approach decouples producers from consumers entirely.

Requirements:

- Events must be modelled as immutable facts with a well-defined, published schema
- Event schemas must be versioned and backward-compatible within a major version
- Consumers must handle duplicate or out-of-order delivery

Applicable Patterns / Technology Guidance:

- Event streaming platform such as Kafka
- Schema registry to govern and version event schemas

6. Non-Functional Requirements

6.1 Observability

Statement: Solutions must emit structured logs, metrics and distributed traces sufficient to diagnose issues without requiring code changes.

Rationale: Without observability, diagnosing production issues becomes slow guesswork.

Requirements:

- Structured logging must be implemented with a correlation identifier propagated across service calls
- Key business and technical metrics must be exposed for monitoring and alerting
- Distributed tracing must be implemented for solutions composed of multiple services

Applicable Patterns / Technology Guidance:

- Centralised log aggregation
- Prometheus and Grafana for metrics
- OpenTelemetry for distributed tracing

6.2 Availability and Recovery

Statement: Solutions must define an availability target and recovery objectives aligned to business criticality, and be architected with redundancy to meet them.

Rationale: Explicit targets ensure the right level of investment in redundancy where it matters.

Requirements:

- An availability target must be documented per solution
- Recovery time objective and recovery point objective must be stated explicitly
- Critical solutions must eliminate single points of failure through redundancy

Applicable Patterns / Technology Guidance:

- Multi-zone deployment for redundancy
- Health checks and automated failover

Criticality	Availability Target	RTO	RPO
Tier 1 · Mission critical	99.95%	1 hour	15 minutes
Tier 2 · Business critical	99.9%	4 hours	1 hour
Tier 3 · Productivity	99.5%	8 hours	4 hours

7. Technology Reference Model

Layer	Purpose	Representative Technologies
Presentation	User-facing interfaces	Component-based web frameworks; native or cross-platform mobile
API & Integration	Exposing and managing service interfaces	API gateway; integration platform; schema registry
Application	Core business processing	Container-based services; managed runtimes; serverless functions
Data & Storage	Persisting and querying data	Relational OLTP; document store; data warehouse; distributed cache
Observability	Monitoring and diagnosing system health	Centralised logging; metrics; distributed tracing; alerting

Appendix A: Glossary of Terms

Term	Definition
ADR	Architecture Decision Record · a short document capturing a significant decision.
Circuit Breaker	A pattern that halts calls to a failing dependency to prevent cascading failure.
RPO	Recovery Point Objective · the maximum acceptable data loss, measured in time.
RTO	Recovery Time Objective · the maximum acceptable time to restore a system.